

AXI + UART Integration	2
1. Title	2
2. Objective	2
3. Theory (Multi-Level)	2
3.1 Beginner — Simple Intuition.....	2
3.2 Intermediate — Engineering View.....	2
3.3 Advanced — Signal/Architecture Level	3
4. Architecture / Concept.....	4
uart_axi Internal Block Diagram	4
AXI-UART Data Flow (Full End-to-End)	5
5. Implementation.....	5
Files in This Module.....	5
Key Code Paths in rtl/uart_axi.v	6
6. Lab / Hands-on Section	6
Step 1: Run AXI-UART Integration Testbench	6
Expected Terminal Output.....	6
Step 2: View AXI-UART Waveform	7
7. Waveform / Verification Insight.....	8
AXI Write → UART TX Timeline.....	8
Backpressure Observation.....	8
RX Path Timeline	8
8. Common Issues & Debug Tips	8
Debug: Trace buf_valid Signal.....	9
9. Summary	9

AXI + UART Integration

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

AXI + UART Integration — UART Peripheral as an AXI-Lite Slave

2. Objective

By the end of this module, learners will be able to:

- Explain how a UART peripheral is exposed as an AXI-Lite slave
- Map the `uart_axi` module's internal registers to AXI addresses
- Trace a complete AXI write → UART TX byte-serialization path
- Trace a complete UART RX → AXI read path
- Run the `make axi_uart` testbench and observe data flow through the AXI-UART pipeline

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

Imagine you want to use a UART peripheral in an AXI-based SoC. The UART speaks “serial bits”, the AXI bus speaks “parallel register reads/writes”. We need an **adapter**:

- You write to AXI address `0x10000000` → the adapter stores the byte and triggers UART TX
- UART receives a byte → the adapter stores it → you read AXI address `0x10000004` to get it
- You read AXI address `0x10000008` → the adapter tells you if new data is waiting

The `uart_axi.v` module is this adapter. Externally it looks like a simple register bank to the AXI bus; internally it drives the `uart_tx.v` and reads from `uart_rx.v`.

3.2 Intermediate — Engineering View

uart_axi.v Register Map:

AXI Address	Access	bit[8]	bits[7:0]	Description
<code>0x10000000</code>	Write	—	TX data	Write byte to transmit
<code>0x10000004</code>	Read	<code>rx_valid</code>	<code>rx_data</code>	Read received byte (clears <code>rx_buf_valid</code>)

AXI Address	Access	bit[8]	bits[7:0]	Description
0x10000008	Read	—	{rx_valid, 1'b0}	Status: bit[1]=rx_buf_valid

Write path (AXI → UART TX):

AXI master write to 0x1000_0000
 → uart_axi captures (aw_got=1, w_got=1)
 → w_byte ← wdata[7:0] (based on wstrb)
 → buf_data ← w_byte; buf_valid ← 1
 → uart_tx gets: data=buf_data, valid=1
 → 8N1 serial bitstream on tx_out pin
 → B response sent back: OKAY (2'b00)

Read path (UART RX → AXI):

uart_rx detects start bit on uart_rx pin
 → uart_rx shifts in 8 bits
 → data_valid pulse → rx_buf_data ← received byte; rx_buf_valid ← 1
 AXI master read from 0x1000_0004
 → uart_axi returns {23'b0, rx_buf_valid, rx_buf_data}
 → rx_buf_valid ← 0 (cleared on data register read)

3.3 Advanced — Signal/Architecture Level

Write backpressure mechanism (ASIC-clean, from rtl/uart_axi.v):

```
// Ready assertions — held LOW when buffer is full
assign s_axi_awready = !aw_got && !buf_valid;
assign s_axi_wready = !w_got && !buf_valid;
```

This ensures that if the 1-byte TX buffer is full (buf_valid=1), the AXI master is **stalled at the write data channel** — AWREADY and WREADY go low, forcing the CPU bridge to wait. This prevents silent byte drops.

TX buffer state machine:

Reset: buf_valid=0, buf_data=0

If tx_ready && buf_valid: → tx_data←buf_data, tx_valid←1, buf_valid←0
 If aw_got && w_got && !b_vld && !buf_valid: → buf_data←w_byte, buf_valid←1

(These two conditions interleave: UART drains buffer)
 (as fast as possible, AXI refills when buffer is empty)

WSTRB byte selection:

```
if (s_axi_wstrb[0]) w_byte <= s_axi_wdata[ 7: 0]; // byte 0
else if (s_axi_wstrb[1]) w_byte <= s_axi_wdata[15: 8]; // byte 1
```

```
else if (s_axi_wstrb[2]) w_byte <= s_axi_wdata[23:16]; // byte 2
else w_byte <= s_axi_wdata[31:24]; // byte 3
```

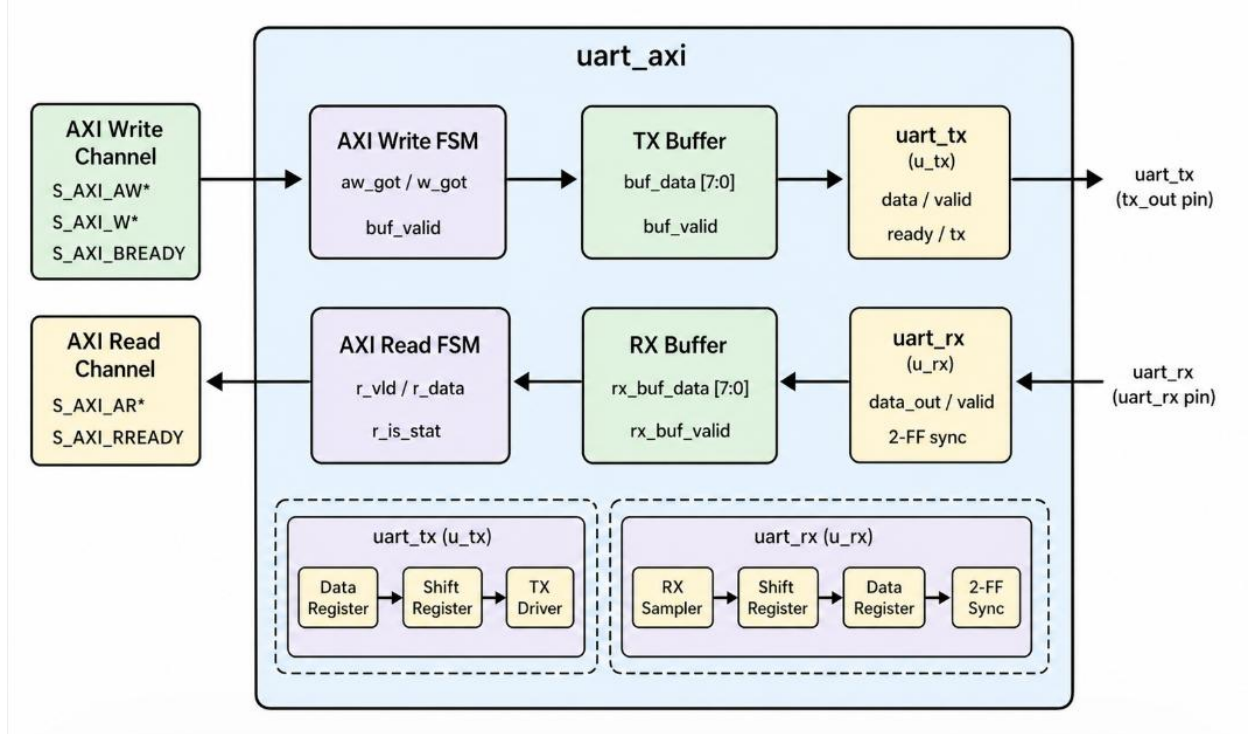
This allows 8-bit firmware stores (SB instruction, wstrb=4'h1) to correctly target the lower byte of the 32-bit AXI data word.

Module parameters:

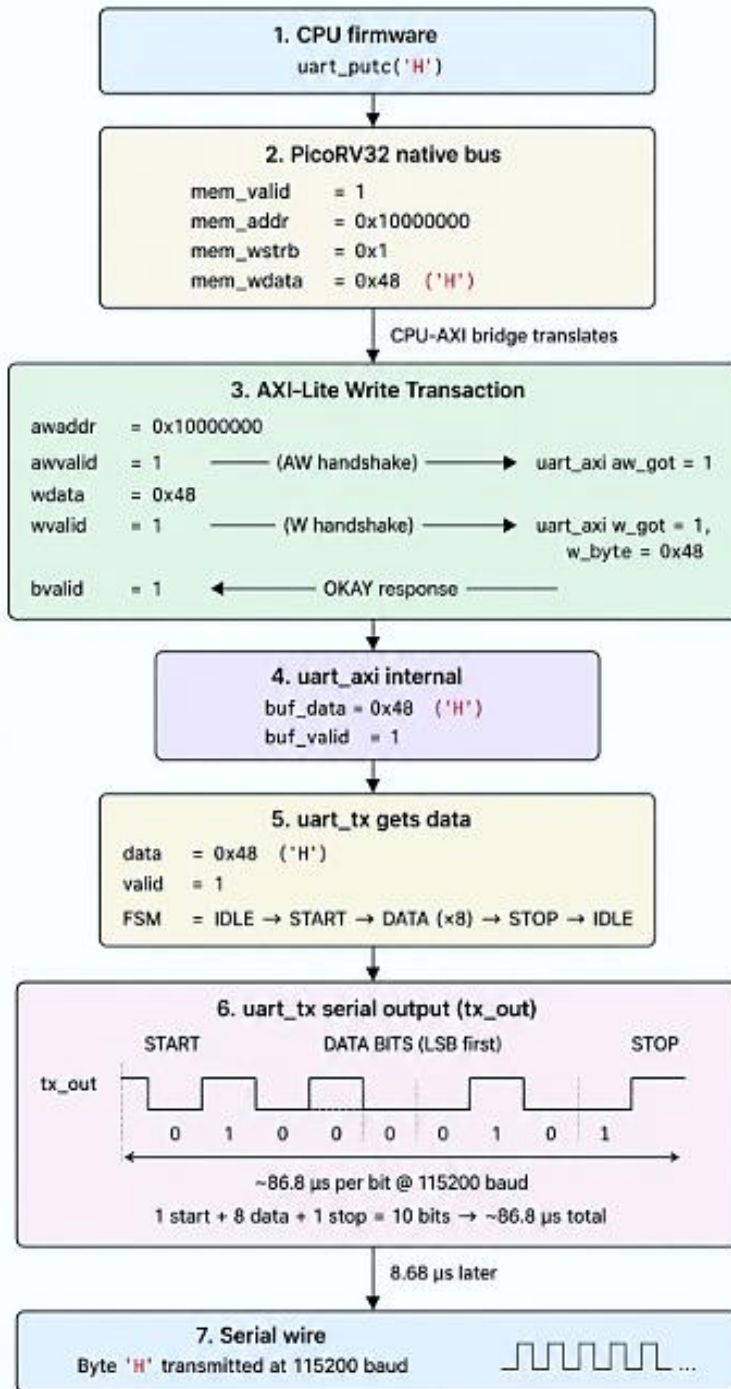
```
module uart_axi #(
    parameter CLK_FREQ = 50_000_000, // System clock
    parameter BAUD_RATE = 115_200 // Serial baud rate
)
// CLKS_PER_BIT = 50_000_000 / 115_200 = 434 cycles per bit
```

4. Architecture / Concept

uart_axi Internal Block Diagram



AXI-UART Data Flow (Full End-to-End)



5. Implementation

Files in This Module

File	Role
rtl/uart_axi.v	UART peripheral as AXI-Lite slave — TX buffer + RX buffer + AXI controller
rtl/uart_tx.v	Instantiated inside uart_axi as u_tx
rtl/uart_rx.v	Instantiated inside uart_axi as u_rx
tb/tb_axi_lite.v	Testbench: AXI master + uart_axi DUT + UART serial monitor
tb/tb_axi_lite_master.v	Reusable AXI master task module

Key Code Paths in rtl/uart_axi.v

Write channel ready conditions (lines 172–173):

```
assign s_axi_awready = !aw_got && !buf_valid;
assign s_axi_wready = !w_got && !buf_valid;
```

RX read data format (lines 237–241):

```
if (s_axi_araddr[3:0] == 4'h8)
    r_data <= {30'b0, rx_buf_valid, 1'b0}; // Status register
else
    r_data <= {23'b0, rx_buf_valid, rx_buf_data}; // Data register:
    [8]=valid, [7:0]=data
```

Port naming note: The TX output is named tx_out (not uart_tx) to avoid Yosys scope-resolution collision with the submodule type name uart_tx. This is a real ASIC synthesis concern documented in the code:

```
output tx_out, // renamed: avoids collision with submodule type `uart_tx`
```

6. Lab / Hands-on Section

Step 1: Run AXI-UART Integration Testbench

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
make axi_uart
```

This: 1. Cleans obj_axilite/ 2. Compiles: uart_tx.v + uart_rx.v + uart_axi.v + tb_axi_lite_master.v + tb_axi_lite.v 3. Runs ./obj_axilite/sim_axilite → produces tb_axi_lite.vcd 4. Opens GTKWave

Expected Terminal Output

AXI-UART Testbench Started

```
[AXI WRITE] addr=0x10000000 data=0x48 ('H') → Writing to UART TX  
[AXI WRITE] awvalid=1 → Sent address  
[AXI WRITE] awready=1 → Address accepted  
[AXI WRITE] wvalid=1 → Sent data 0x00000048  
[AXI WRITE] wready=1 → Data accepted  
[AXI WRITE] bvalid=1 → Response: OKAY
```

```
[UART TX] Transmitting: 0x48 ('H')  
[UART TX] Start bit → 8 data bits → Stop bit
```

```
[AXI READ] addr=0x10000008 → Reading UART status  
[AXI READ] rdata=0x00000002 → RX data available
```

```
[AXI READ] addr=0x10000004 → Reading UART RX data  
[AXI READ] rdata=0x00000148 → rx_valid=1, rx_data=0x48 ('H')  
[TEST] Loopback data matches → PASS
```

Step 2: View AXI-UART Waveform

gtkwave tb_axi_lite.vcd &

Add these signals in GTKWave:

clk ← System clock
reset

```
# AXI Write channel  
s_axi_awaddr[31:0]  
s_axi_awvalid  
s_axi_awready ← Stalls when buf_valid=1  
s_axi_wdata[31:0]  
s_axi_wvalid  
s_axi_wready  
s_axi_bvalid  
s_axi_bready
```

```
# AXI Read channel  
s_axi_araddr[31:0]  
s_axi_arvalid  
s_axi_arready  
s_axi_rdata[31:0]  
s_axi_rvalid  
s_axi_rready
```

```
# UART TX internals
```

```
u_tx.tx ← Serial output (observe 8N1 frame)  
u_tx.state ← IDLE/START/DATA/STOP
```

`u_tx.clk_cnt[9:0]` ← Baud rate counter (0-433)

UART buffer

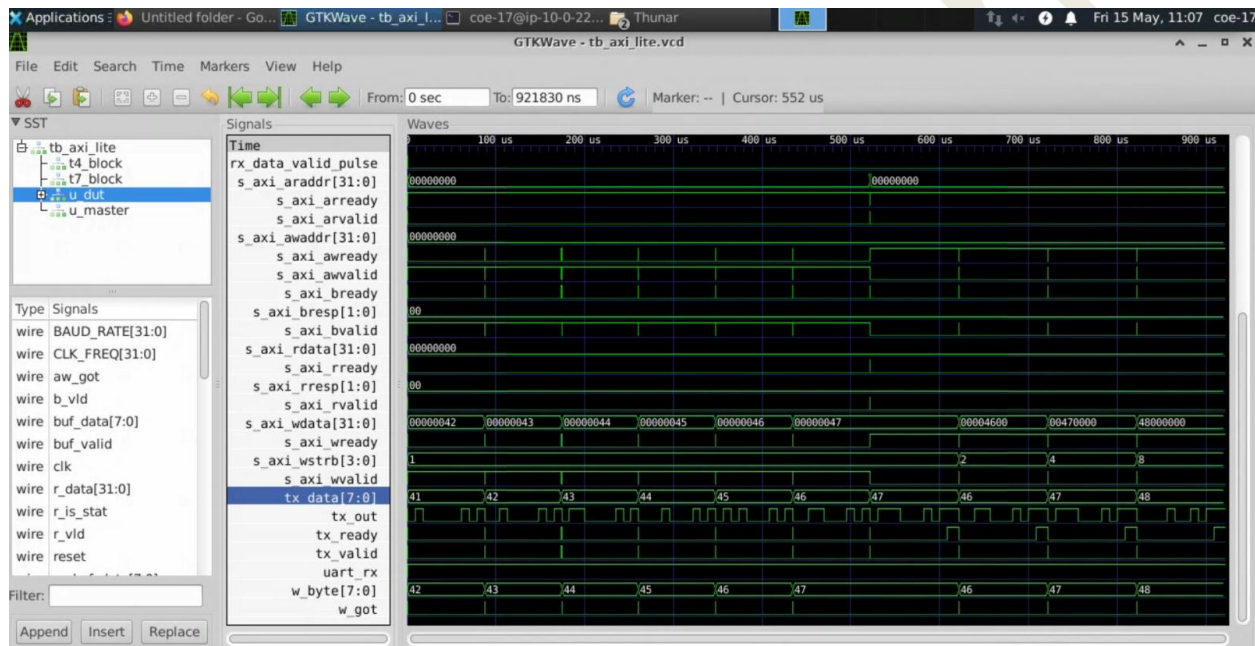
`uart_axi.buf_valid` ← TX buffer occupancy

`uart_axi.aw_got` ← AW channel captured

`uart_axi.w_got` ← W channel captured

`uart_axi.rx_buf_valid` ← RX data waiting

7. Waveform / Verification Insight



AXI Write → UART TX Timeline

In GTKWave, look for this sequence: 1. `s_axi_awvalid=1` (master presents write address) 2. `s_axi_awready=1` (AW handshake) — `uart_axi.aw_got` pulses high 3. `s_axi_wvalid=1` → `s_axi_wready=1` (W handshake) — `uart_axi.w_got` pulses high 4. `s_axi_bvalid=1` (B response OKAY) 5. `uart_axi.buf_valid=1` → triggers `uart_tx.valid=1` 6. `u_tx.tx` drops to 0 (start bit) → 8 data bits → returns to 1 (stop bit)

Backpressure Observation

Send two bytes back-to-back and observe: - First byte: `awready=1`, `wready=1` immediately - Second byte (during first TX): `awready=0`, `wready=0` while `buf_valid=1` - Once `uart_tx.ready=1` (UART idle), `buf_valid=0` → `awready` and `wready` return to 1

This backpressure mechanism prevents data loss.

RX Path Timeline

1. uart_rx detects falling edge on uart_rx pin
2. After 4340 cycles, data_valid=1 pulses
3. uart_axi.rx_buf_valid=1, uart_axi.rx_buf_data = received_byte
4. AXI read of 0x10000004 → rdata = {23'b0, 1'b1, received_byte}
5. rx_buf_valid clears after the read handshake

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
UART TX never transmits	buf_valid never goes high	Confirm AW+W handshake completed before checking
UART TX sends wrong byte	w_byte used wrong wstrb lane	Check which WSTRB bit is 1 in your test vector
AXI write stalls permanently	buf_valid stuck high	Check uart_tx.ready — is UART TX FSM completing?
awready stays low forever	aw_got is stuck 1	Did B-channel response complete? Check b_vld
RX data is 0x00 always	RX loopback not connected in TB	Check uart_axi.uart_rx is connected to TX in loopback TB
Status register reads 0	RX byte not yet received	Add delay before status read (UART byte = 86.8 μs at 115200)
tx_out port name issue	Yosys collision with uart_tx module type	Already fixed: port renamed tx_out in rtl/uart_axi.v

Debug: Trace buf_valid Signal

// Add in testbench for live debug:

```
always @(posedge clk) begin
    if (dut.buf_valid && dut.tx_ready)
        $display("[DBG] TX byte sent: 0x%02h", dut.buf_data);
    if (dut.rx_buf_valid)
```

```
end      $display("[DBG] RX byte ready: 0x%02h", dut.rx_buf_data);
```

9. Summary

Concept	Detail
uart_axi role	Wraps uart_tx+uart_rx with AXI-Lite slave interface
TX write addr	0x10000000 → byte from wdata[7:0] → 1-byte buffer → uart_tx
RX data addr	0x10000004 → returns {rx_valid[8], rx_data[7:0]}
Status addr	0x10000008 → returns {rx_valid[1], 1'b0}
Backpressure	awready=wready=0 when buf_valid=1 — zero data loss
Port naming	tx_out (not uart_tx) to avoid Yosys scope collision
ASIC-clean	No inline reg initializers; all state reset-driven
Lab result	AXI write → UART byte serialized; RX loopback → AXI read verified

This module provides the complete “last mile” between the AXI bus and the physical UART serial lines. Combined with Sub-module 5 (CPU-AXI bridge) and Sub-module 7 (firmware), this forms the complete software-to-wire data path of the SoC.

Previous: Sub-module 5 — RISC-V AXI Integration

Next: Sub-module 7 — Firmware Implementation